Colegiul Naţional “Moise Nicoară” Arad

LUCRARE DE ATESTAT
DISCIPLINA INFORMATICĂ

Candidat,
Otlăcan Fabian,
Bornea Răzvan

Coordonator,
Prof. Cameniță Daniela
Prof. Gal Carmen

2026

Platforma de pregătire pentru examene

CUPRINS

1. Introducere 4
1.1. Justificarea temei 4
1.2. Necesități soft-hard 4
1.3. Programele utiliziate 4
1.4. Rularea programului 5
2. Partea teoretică 6
3. Partea practică 8
3.1. Interfața grafică 8
3.2. Subiectele 9
3.3. Gestionarea bazei de date 11
3.4. Grafice 12
3.5. Concluzii 14
4. Bibliografie 15

1. Introducere

1.1. Justificarea temei

Am ales această temă din dorința de a centraliza subiectele de admitere de la diverse universitățăti cu posibilitatea de evaluare automată, deoarce la momentul actual acestea trebuie căutate individual, iar în unele cazuri această sarcină nu este una ușoară, iar opțiunea de evaluare atuomată nu este mereu posibilă. Din păcate sunt disponibile doar subiecte de tip grilă, acestea fiind ușor de corectat automat.

Proiectul nu se rezumă doar la subiecte de admitere. Datorită modularității sistemului de gestionare a subiectelor, oricine poate crea un subiect propriu și îl poate da mai departe oricui.

1.2. Necesități soft-hard

Necesități software:

- interpretator python3
- librăriile matplotlib, pyqt5, sqlite3, threading, sockets, os, hashlib, datetime, sys, json, PIL
- sqlite3
- sistem de operare Windows 10/11 sau Linux
- access la interfață în linie de comandă

Necesități minime de hardware:

- Procesor x86-64 Dual Core
- Placă video ce suportă DirectX
- 4 Gb ram
- Display de 1024x768

1.3. Programele utiliziate

Pentru a scrierea codului am folosit editorul de text VIM si IDE-ul Visual Studio Code, iar pentru rularea acestuia am folosit interpretatorul python3 în consolă.

Pentru realizarea diverselor meniuri am folosit aplicația QtDesigner care oferă o interfață intuitivă cu acest scop.

1.4. Rularea programului

Pentru a rula programul este nevoie să porniți mai întâi serverul. Pentru acest lucru navigați în folderul /server în terminal și rulați comanda „$python3 server.py”. Acest lucru este necesar pentru a vă putea loga în aplicație, altfel nu puteți comunica cu baza de date.

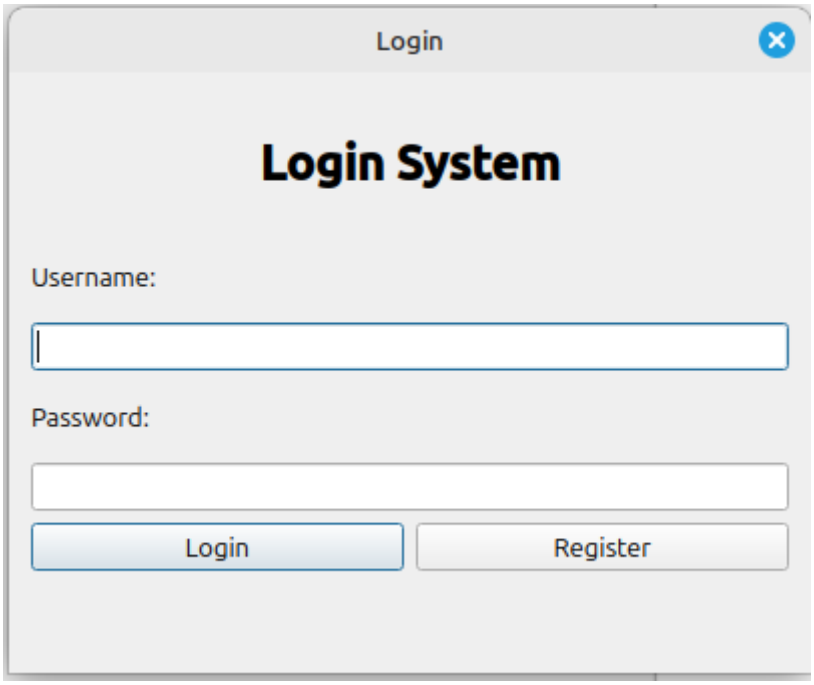
După ce serverul rulează, putem deschide aplicația principală navigând în folderiul /client și rulând comanda „$python3 main.py”. Din acest moment aveți acces la o interfață grafică intuitivă și ușor de utilizat.

Pentru a închide serverul folosiți combinația de taste CTRL + C în terminalul în care rulează serverul.

Spre dacă fișerul Atestat are calea Dev/Atestat, vom folosi comenzile următoare:

```
fabi@fpc:~$ cd Dev/Atestat/server/
fabi@fpc:~/Dev/Atestat/server$ python3 server.py
SERVER STARTED
```

```
fabi@fpc:~$ cd Dev/Atestat/client/
fabi@fpc:~/Dev/Atestat/client$ python3 main.py
```



2. Partea teoretică

Pentru a realiza interfața grafică a programului am folosit librăria PyQt care oferă definiții pentru orice element de design de care am avut nevoie: layout-uri, tabele, butoane, etichete, ferestre și meniuri. În locurile unde am avut nevoie de o formatare specială am putut folosi StyleSheets CSS, care oferă o flexibilitate suficientă pentru orice am avut nevoie.

În ceea ce privește subiectele de pe platformă, am fost nevoiți să implementăm singuri toată logica din spatele acestora, folosindu-ne doar de elemente ale limbajului python și fișiere json pentru a stoca și citi informațiile. Folosirea fișierelor json face foarte ușoară modificarea unui subiect, ceea ce oferă modularitate acestui sistem creat de la zero de noi.

Conturile utilizatorilor sunt stocate într-o bază de date gestionată folosind SQLite. Am alest această librărie din două motive:

(a) Numar limitat de feature-uri: aplicația noastră nu are nevoie de multe feature-uri, ci de simplitate, din moment ce stocăm un număr mic de informații

(b) Baza de date ca fișier: nu dorim să complicăm proiectul mai mult decât este nevoie, iar cu ajutorul sqlite putem păstra baza de date într-un singur fișier, comparativ cu alte librării cum ar fi MySql care este mult mai complexă

Comenziile din SQLite de care am avut nevoie sunt:

CREATE TABLE – pentru crearea unui tabel de utilizatori, respectiv teste

SELECT name FROM table WHERE name=„nume” –pentru a căuta informații într-un tabel

INSERT INTO table VALUES() - pentru a adăuga o linie într-un tabel

UPDATE table SET value = new_value WHERE name = „nume” – pentru a actualiza o valuare asociată unui utilizator

Pentru comunicarea server-client folosim librăria socket, în care avem acces direct la tot ce se întâmplă în procesul de comunicare, în aspecte cum ar fi portul folosit, mijlocul de encodare și headerul folosit.

Pentru criptarea parolelor se folosește algoritmul de hashing sha256 din librăria hashlib.

Serverul este momentan local pentru a reduce din problemele care pot să apară în timpul prezentării. Pentru a îl face disponibil la nivel de rețea este nevoie de modificări minore în codul sursă, iar pentru a-l face disponibil prin internet este nevoie de un domeniu public.

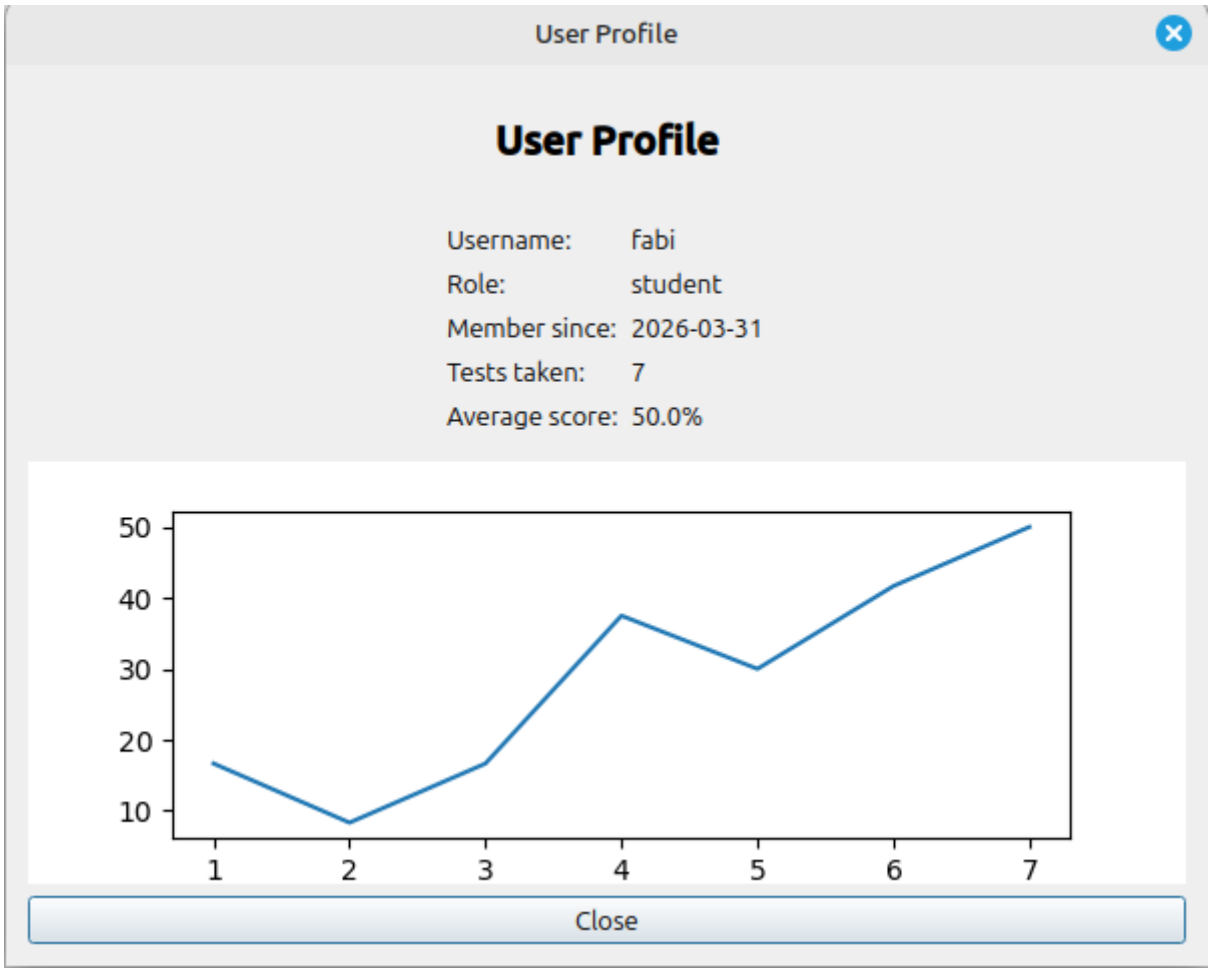
Pentru crearea graficelor am folosit următoarele funcții și obiecte din matplotlib:

a) matplotlib.backends.backend_qtagg (importat sub numele de FigureCanvas) care folosește la integrarea graficelor din matplotlib în interfața PyQt

b) Figure care este graficul propriu zis

c) Figure.plot(x, y) care ia doua liste de numere reale de aceeași lungime și formează un grafic cu punctele de tipul Pi(x[i], y[i])

d) FigureCanvas.draw() care actualizează graficul după ce a fost deja adăugat în layout



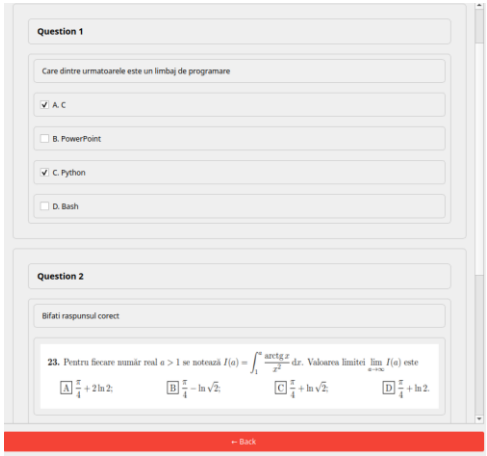
3. Partea practică

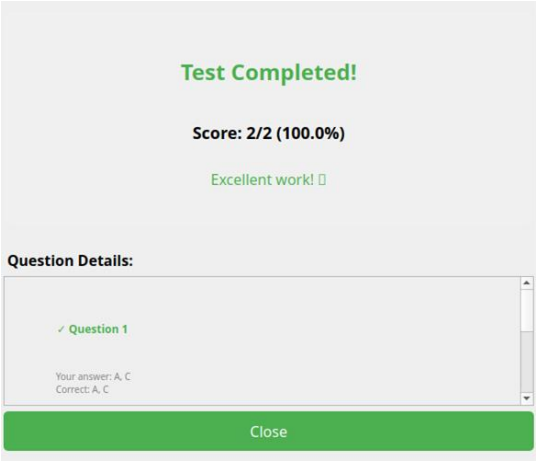
3.1. Interfața grafică

Chiar dacă PyQt ne oferă elementele de bază pentru a construi interfața grafică, avem nevoie de un sistem care sa permită extinderea acesteia rapid, deoarce scopul este sa avem cât mai multe subiecte. Am realizat acest lucru folosind un folder pe care programul îl folosește pentru a își crea automat interfața după următorul alogritm: scanează fiecare fișier din folderul curent, daca este de format .json creaza un buton pentru a deschide un subiect, altfel creaza un buton pentru a deschide un nou folder (orice alt format nu este permis).

O dată ce un buton este apăsat, fie se deschide un subiect de examen, fie se apeleaza recursiv algoritmul descris mai sus pentru folderul selectat de către utlizator. Pentru a ști în ce folder se află la fiecare moment, programul salvează acestă informație într-o variabilă globala numită path. Toată logica descrisă mai sus este implementată în clasa Pagina din fișerul objects.py.

Tot în fișierul objects.py avem implementările pentru clasele de subiect și exericiţiu, unde am definit funcțiile de afișare, evaluare, deschidere și trimitere. Exercițiul conține informații despre textul problemei, variantele corecte de răspuns și variantele de răspuns alese de utilizator, iar subiectul conține o listă de exerciții.





Așezarea în pagină a butoanelor se face folosind un VerticalLayout care le așează în mod automat unul deasupra celuilalt. Acest tip de layout este folosit și în interiorul unui subiect pentru a afișa exercitiile.

Pentru meniurile ce apar o singură dată am scris o funcție care sa le randeze în fișierul objects.py sau interface.py, în funcție de dependențele de alte clase/funcții. Pentru a realiza mai rapid aceste meniuri am folosit aplicația QtDesigner, care permite generarea codului în mod interactiv prin drag and drop (similar cu form-urile din visual studio pentru C#).

Anumite meniuri sunt disponibile doar utilizatorilor cu rolul de admin sau de profesor. Restricționarea se realizează cu ajutorul unor simple structuri decizionale.

3.2. Subiectele

Structura unui exercițiu este:

Enunț: un șir de caractere care reprezintă enunțul exercițiului

Imagini: O lista cu calea spre fiecare imagine

Răspunsuri: Un dicționar cu răspunsurile corecte

Options: Un dicționar cu răspunsurile disponibile

Structura unui subiect este:

titlu: un șir de caractere care reprezintă titlul subiectului

Exerciții: un vector de exerciții (structura definită mai sus)

Un exemplu de subiect este:

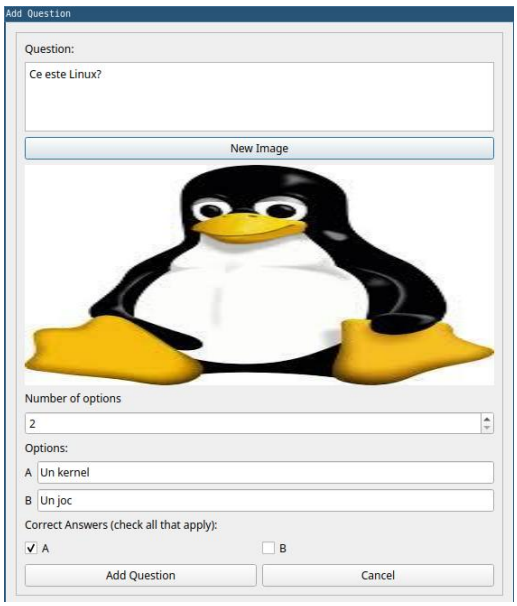
```
{
  "titlu": "Test",
  "Exercitii": {
    "1": {
      "Enunt": "Care dintre urmatoarele este un limbaj de programare",
      "Imagini": [],
```

"Raspunsuri": ["A", "C"],
"Options": {
"A": "C",
"B": "PowerPoint",
"C": "Python",
"D": "Bash"} },
"2": {
"Enunt": "Bifati raspunsul corect",
"Imagini": ["Images/imagine.png"],
"Raspunsuri": ["B"],
"Options": {
"A": "",
"B": "",
"C": "",
"D": "",
"E": "",
"F": ""
}}}}

Observație: Dacă un câmp este lăsat liber, acesta este completat automat cu „Option A/B/C/...”

La parsare fiecare optiune devine un buton și enunțul devine o etichetă.

La crearea unui nou test de către utlilizator este nevoie doar să cerem de la utilizator informațiile necesare despre fiecare exercițiu în parte. Folosim căsuțe de text pentru enunț și opțiunile disponibile și check box-uri pentru răspunsurile corecte, astfel toate informațiile vor fi conforme cu formatul impus în fișierele json.



3.3. Gestionarea bazei de date

Baza de date se află în fișierul /server/database.db care este gestionat din server.py. Din moment ce utilizatorul folosește aplicația executată folosind /client/main.py, avem nevoie de un protocol de comunicare între cele două pentru a gestiona conturile utilizatorilor din interfața grafică.

Procesul de comunicare dintre client și server se desfășoară astfel: clientul trimite un request definit de noi (la bază fiind doar un sir de caractere) pe care serverul îl parsează și execută comanda dacă aceasta este validă, trimițând un răspuns înapoi clientului. Răspunsul poate fii „done” în cazul în care comanda nu cere un output, sau outputul cerut în caz contrar.

Comenziile pe care serverul le acceptă la momentul actual sunt:

- CREATEUSER
- AUTHUSER (authentifica utilizatorul)
- GETUSER (trimite informații despre un utilizator)
- UPDATEUSER (schimbă informații despre utilizator)
- GETALLUSERS (trimite informații despre toți utilizatorii)
- SAVEANALYTICS (salvează rezultatele unui test)
- GETANALYTICS (procesează rezultatele unui test)
- GETTESTSBYUSER (trimite rezultatele testelor unui anumit utilizator)

Ultimele două comenzi sunt relevante doar în contul de profesor, care poate vedea toate rezultatele fiecărui test susținut de un utilizator.

Gestionarea conturilor se realizează în fișierul auth.py pe partea clientului și în fișierul server.py pe partea serverului. Funcțiile send() și recv() sunt implementari standard pentru a trimite și a primi un mesaj dintr-o conexiune.

Functia parse() primește un string și returneaza o pereche de tipul (primul cuvânt din s, s fără primul cuvânt). Aceasta funcție este folosită pentru a prelucra pe rând fiecare parametru dintr-o comandă primită de către server.

Atunci când aplicația client dorește sa acceseze baza de date va trimite o comanda în următorul format către server: „COMANDA_PARAM1_PARAM2_..._PARAMN_” unde COMANDA este numele comenzii, PARAMN = al n-lea parametru, iar n este numărul de parametrii. Un exemplu de astfel de comandă este GETUSER_Gigel_ care ia toate informațiile despre utilizatorul Gigel, dacă acesta există.

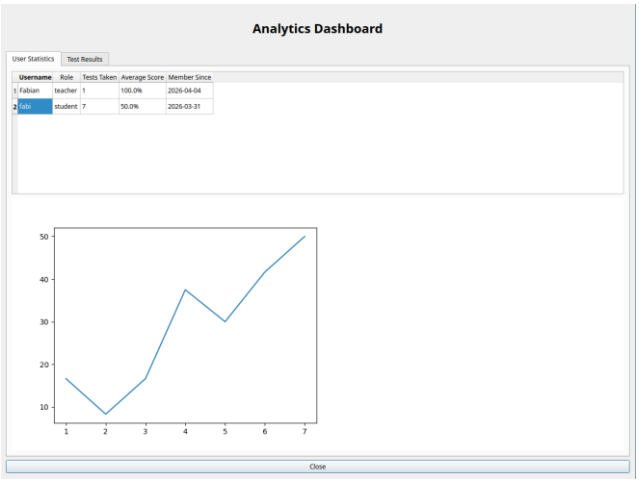
Folosim caracterul ‘_’ pe post de separator deoarece spațiul ar fi confundat de către server ca o comandă separată. Caracterul final ‘_’ este necesar pentru a fi adăugat ultimul parametru în comandă, deoarece așa am implementat funcția parse().

Pentru criptarea parolelor se folosește algoritmul de hashing sha256 din librăria hashlib. Aceasta este una dintre multele erori de securitate pe care proiectul nostru ar fi nevoit să le adreseze, o alta fiind SQL injection-urile.

În momentul în care serverul este pornit, acesta așteaptă să i se ceară o conexiune. În momentul când conexiunea este stabilită deschide un nou thread în care așteaptă comenzi de la client, iar când acesta primește comanda exit închide conexiunea. Thread-urile sunt folosite pentru a gestiona mai mulți utilizatori simultan. Acest lucru este realizat folsind librăria threading din limbajul python.

3.4. Grafice

Pentru crearea graficelor cu progresul unui utilizator am folosit librăria matplotlib. Deoarce nu putem stoca prea multe date despre un utilizator în baza de date, am cerut o listă cu toate testele susținute de utilizator din baza de date și pe baza acestor informații am calculat procentul de reușită mediu după fiecare test susținut, adaugând un punct pe grafic. Acest lucru a fost realizat cu ajutorul comenzii GETTESTSBYUSER.



Această parte de logică se găsește în fișierul interface.py în funcția get_user_progress_graph(), funcție folosită mai departe în meniurile de profil și tabelul de utilizatori.

4. Concluzii

Proiectul reprezintă o bază solidă pentru o aplicație ce poate ajuta elevii și profesorii în diverse situații, cum ar fi pregătirea pentru un examen de admitere, dar totuși încă nu este completă. Câteva exemple de imbunătățiri necesare sunt: un server central, securitate mai bună în ceea ce privește criptarea parolele utilizatorilor, gestionarea în siguranță a comenzilor SQL și numărul de subiecte disponibile la instalare. De asemenea, experiența de utilizare poate fi îmbunătățită pentru a permite oricui să folosească aplicația, fără a avea nevoie de un interpretator python.

În ceea ce privește crearea proiectului, acesta a fost o experiență din care am învățat multe lucruri, de la integrarea unei baze de date într-un program real la stabilirea unei conexiuni între două procese de pe calculatoare diferite (pentru simplitate am păstrat totul pe un singur calculator în lucrarea finală). Am avut de învățat și pe partea umană, fiind nevoiți să înțelegem ideile celuilalt și să alegem varianta cea mai bună atunci când nu eram întru totul de acord unul cu celălalt. Împarțirea sarcinilor a fost și ea o provocare, deoarce nu toate au fost la fel de interesante, unele fiind chiar plictisitoare și a trebuit să găsim un mod de a le împarți în mod egal.

În concluzie, proiectul este unul reușit, chiar dacă nu complet, îndeplinidu-și scopul și oferindu-ne posibilitatea de a avansa în privința cunoștiintelor tehnice și a abilităților de colaborare.

5. Bibliografie

Nu am avut nevoie de documentație/tutorial python, amândoi fiind deja familiari cu limbajul

Documentatia pyqt: https://doc.qt.io/qtforpython-6/

Folosirea sockets și threading: https://www.youtube.com/watch?v=3QiPPX-KeSc&t=2733s

SQL tutorial: https://www.w3schools.com/sql/

Sqlite în python: https://docs.python.org/3/library/sqlite3.html

https://www.youtube.com/watch?v=byHcYRpMgI4&t=2871s

Grafice: https://www.youtube.com/watch?v=AHhcwFPQlfQ

Pentru subiecte:

https://www.cs.ubbcluj.ro/admitere/nivel-licenta/subiecte-din-anii-precedenti/